

Videoflow

A directed-graph framework for distributed video and stream processing: build the pipeline once, run it as subprocesses or as Kubernetes workloads

White paper · Videoflow 1.0.0 · July 2026

Repository: github.com/videoflow/videoflow

ABSTRACT

Videoflow is a Python framework for building video and stream processing applications as a directed acyclic graph of *producers*, *processors*, and *consumers*. Its central design decision is that **the graph is built on one machine and executed on many**: each node is serialized to a (class path, params) record and reconstructed inside its own worker — an OS subprocess during development, a Kubernetes workload in production — with all inter-node traffic carried by per-node NATS JetStream streams. One retention-policy switch turns the same graph from a loss-free, backpressured batch pipeline into a freshest-wins real-time pipeline. This paper describes the programming model, the compilation pipeline, the messaging topology that yields broadcast, competing-consumer, and partitioned routing purely through consumer naming, the dual wire format (msgpack and protobuf) that enables components written in any language, and the deployment tooling. We evaluate the runtime with eight experiments on toy graphs: wire-format microbenchmarks, horizontal scaling of a stage to 8 replicas (78 % of ideal), per-hop pipeline overhead (≈ 2.0 ms per stage), delivery semantics under $2.5\times$ overload (BATCH delivers 100 % with growing latency; REALTIME delivers 42 % with flat ≈ 171 ms latency), partitioned join scaling, quorum-based fusion of unsynchronized sources, a payload-size sweep that locates the exact point where broker-inline transport fails and the Redis blob store becomes necessary (1,024,000 B passes, 1,048,576 B is rejected), and a fault-injection run in which a SIGKILLED worker replica causes zero loss and zero duplicates.

Contents

- | | |
|---|---------------------------|
| 1 Introduction | 6 The wire format |
| 2 Programming model | 7 Deployment |
| 3 Architecture: build once, run anywhere | 8 Extension seams |
| 4 Messaging topology and delivery semantics | 9 Experimental evaluation |
| 5 Joins | 10 Related work |

11 Conclusion

1 Introduction

Video-analytics applications are naturally pipelines: decode a stream, detect objects, track them, aggregate events, and emit results. Writing such an application as a single process is easy; scaling it past one machine usually is not. The stage that runs a neural network needs a GPU and five replicas, the tracker is stateful and must see every frame in order, the sink calls an external API and must not double-apply effects on retry — and none of that should force a rewrite of the pipeline logic.

Videoflow's answer is to make the pipeline description itself the deployment artifact. A user wires plain Python objects into a DAG and hands the leaves to a `Flow`. The framework validates the graph, compiles every node into a small JSON-serializable record, and starts one worker per node replica. Workers never share memory: every edge of the graph is a subject on a NATS JetStream broker [22], so the same graph runs unchanged as local subprocesses (`LocalProcessEngine`) or as one container per node on Kubernetes (`KubernetesExecutionEngine`). The claim this paper defends is that a small set of conventions — JSON-serializable constructor arguments, I/O-free constructors, name-scoped broker subjects, and ack-after-process delivery — is enough to make a laptop prototype and a multi-machine deployment literally the same program.

Sections 2–8 describe the system: the programming model and its reconstruction contract, the compilation pipeline, the messaging topology and its delivery guarantees, join semantics, the wire format, and the deployment tooling. Section 9 reports six experiments run on the local engine against a real broker, measuring the overheads and semantics the design promises.

2 Programming model

A pipeline is expressed with three node roles, wired by call syntax: `child(parent_a, parent_b)`. Producers are not passed to the flow explicitly — they are discovered by walking parent links backwards from the consumers, and any parentless node that is not a producer is a build-time error.

Role	Base class	Implements	Notes
Producer	<code>ProducerNode</code>	<code>next()</code>	Creates data from an external source; raises <code>StopIteration</code> when finite. <code>is_finite=False</code> marks live sources (RTSP) and selects <code>Deployment</code> vs <code>Job</code> on Kubernetes.
Processor	<code>ProcessorNode</code>	<code>process(*inputs)</code>	One positional argument per parent, in call-site order. Scaling knobs: <code>nb_tasks</code> , <code>partition_by</code> , <code>device_type</code> , <code>gpu_count</code> .
Stateful processor	<code>OneTaskProcessorNode</code>	<code>process(*inputs)</code>	Forces <code>nb_tasks=1</code> ; for trackers, aggregators, and other order-sensitive state.
Consumer	<code>ConsumerNode</code>	<code>consume(item)</code>	Terminal sink; may be declared <code>idempotent=True</code> to deduplicate side effects across redeliveries.

Table 1. The node roles. Every node also has `open()/close()` lifecycle hooks and a stable `name` that is its identity in broker subjects, Kubernetes resources, and logs.

```
from videoflow.core import Flow
from videoflow.core.constants import BATCH
```

```

from videoflow.core.node import ProcessorNode

class Threshold(ProcessorNode):
    def __init__(self, cutoff, **kwargs):
        # args must be JSON-serializable
        self._cutoff = cutoff          # stored verbatim: get_params() finds it
        super().__init__(**kwargs)

    def open(self):
        # heavy setup runs in the worker,
        # never on the machine building the graph
        ...

    def process(self, value):
        return value if value >= self._cutoff else 0

def build_flow():
    producer = FrameProducer('rtsp://...', name = 'camera')
    detector = Detector(name = 'detector', nb_tasks = 4, device_type = 'gpu')(producer)
    gate     = Threshold(0.5, name = 'gate')(detector)
    sink     = ResultsWriter('out.jsonl', name = 'sink')(gate)
    return Flow([sink], flow_type = BATCH)

```

2.1 The reconstruction contract

The rule that makes distribution work — and the one that breaks the most code written against a purely local mental model — is that a worker rebuilds its node from configuration alone, via `type(node)(**node.get_params())`. The default `get_params()` walks every class in the MRO that declares an `__init__`, reads the signature's parameter names, and looks up `self._<name>` (then `self.<name>`). The consequences are mechanical:

- Every constructor argument is stored verbatim under the same name; a missing attribute is an `AttributeError` at build time, not a runtime surprise in a pod.
- Every parameter is JSON-serializable — no node references, file handles, or models.
- `__init__` performs no I/O. It runs on the operator's machine, which may lack the GPU, the weights, or the video file; resources are acquired in `open()` inside the worker and released in `close()`.

Node methods may optionally declare a `ctx` parameter to receive a runtime context (`run_id`, `node_name`, `replica_id`, per-parent input metadata, `set_event_timestamp()`, `set_partition_key()`); the task layer injects it only when the signature asks. `async def` node methods are awaited on a task-owned event loop, keeping node work off the broker I/O loop.

3 Architecture: build once, run anywhere

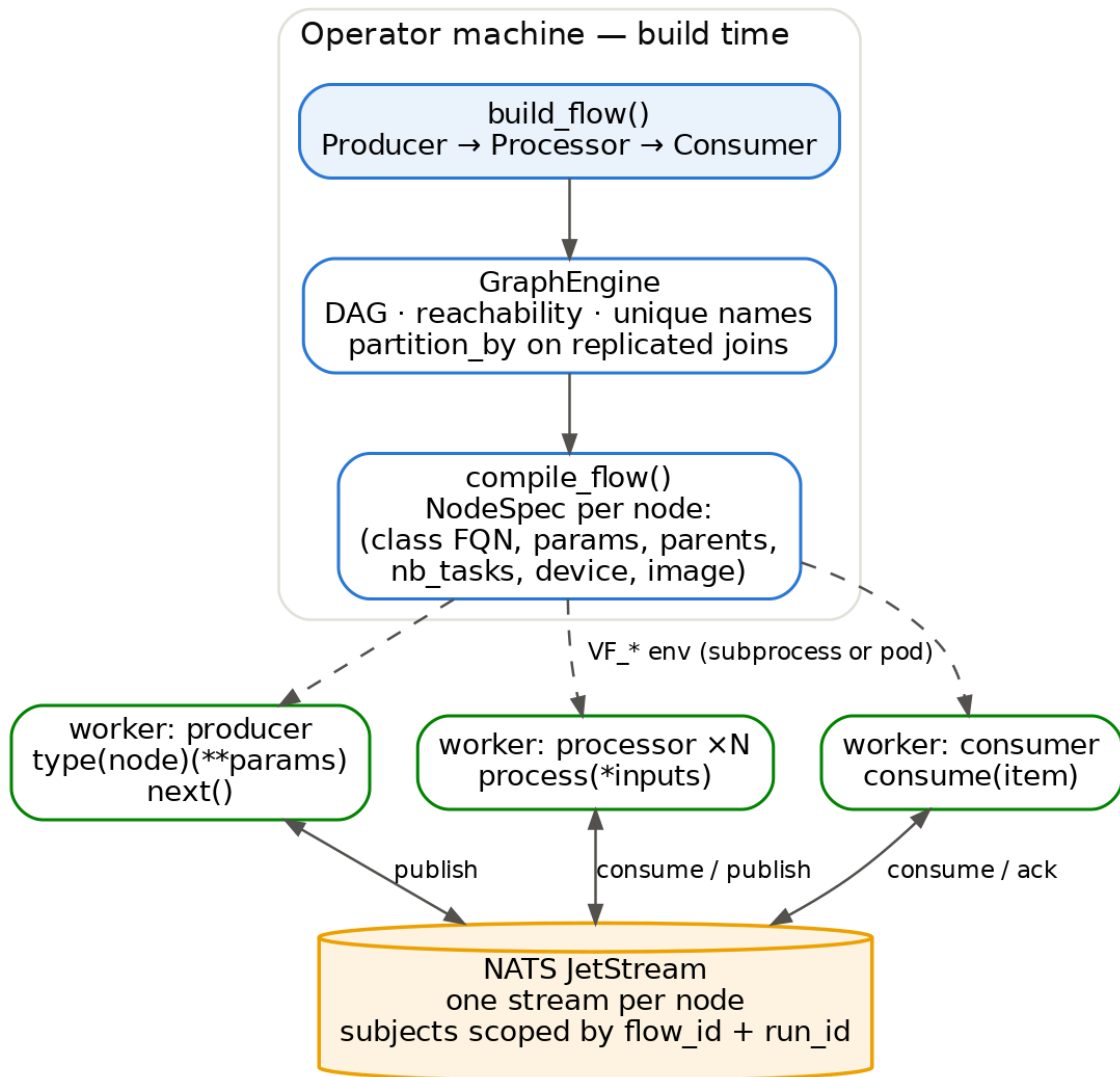


Figure 1. From `build_flow()` to running workers. The graph exists as live objects only on the operator's machine; workers receive (class path, params, parent names) through environment variables and communicate exclusively through per-node broker streams.

Execution proceeds in four steps. (1) `Flow` discovers producers and validates the graph: acyclicity, consumer reachability, unique node names, and — for any replicated multi-parent node — a declared `partition_by`. (2) The compiler flattens the graph in topological order into one `NodeSpec` per node: name, class path, params, parent names, replica count, device requirements, image, join policy. (3) An execution engine provisions the broker topology up front (§4) and starts one worker per replica, passing the spec as `VF_*` environment variables. (4) Each worker (`videoflow.runtime.worker`) reconstructs its node, calls `open()`, and enters its task loop. The worker is byte-for-byte identical between a local subprocess and a Kubernetes pod; only the process launcher differs.

The task loop encodes the delivery semantics. A processor blocks for a complete input group, reorders it to match the declared parent order, calls `process(*inputs)`, publishes the output, and only then acknowledges its inputs — so a crash mid-processing causes redelivery rather than loss. A raised exception fails the inputs (redelivery, then dead-letter in BATCH; drop in REALTIME) without killing the worker: a poison message cannot take down a pod.

4 Messaging topology and delivery semantics

All broker naming lives in a single module (`messaging/topology.py`), and every name is scoped by both `flow_id` and `run_id`, so re-deploying a flow gets fresh streams instead of colliding with a previous run's state.

Object	Name
Data subject	<code>vf.{flow_id}.{run_id}.{node_name}</code>
End-of-stream subject	<code>vf.{flow_id}.{run_id}.{node_name}._eos</code>
Control (stop) subject	<code>vf.{flow_id}.{run_id}._control.stop</code>
Stream (one per node)	<code>vf-{flow_id}-{run_id}-{node_name}</code>
Durable consumer	<code>{consumer_node}--from--{parent_node}</code> (per-replica suffix <code>--p{n}</code> when partitioned)
Dead-letter subject	<code>vf.{flow_id}.{run_id}._dlq.{node_name}</code>

Table 2. Broker naming. The durable-consumer naming alone produces the three routing behaviors in Figure 2.

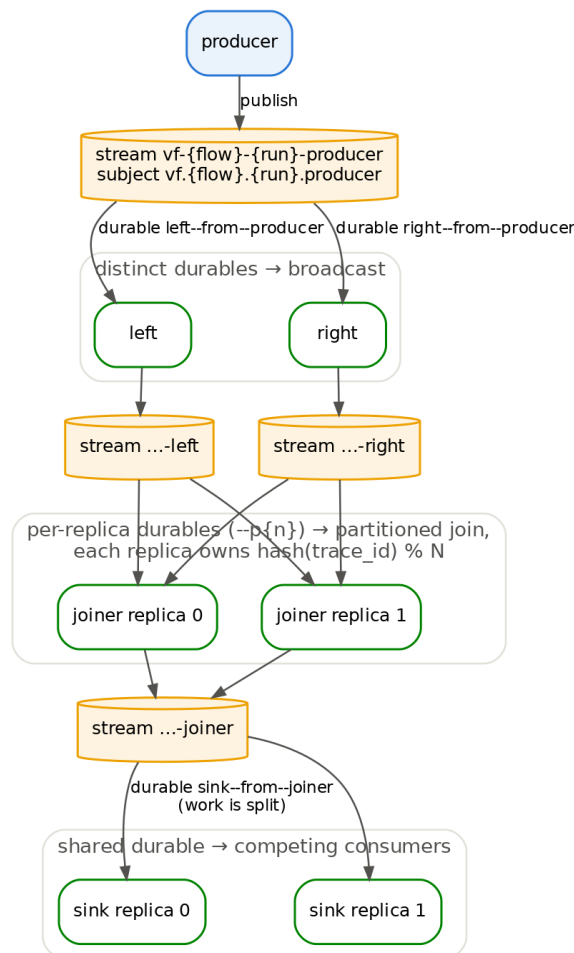


Figure 2. Routing from naming, for a diamond graph with a partitioned join and a replicated sink. Distinct children of one parent get *distinct* durables and therefore each see every message (broadcast); replicas of one node *share* a durable and compete for work; partitioned replicas get per-replica durables and keep only the keys they own (`hash(key) % N`). End-of-stream rides a separate subject so data consumers never see it and every replica can observe it.

Property	BATCH	REALTIME
Retention	Interest-based; bounded backlog	Limits; <code>max_msgs=1</code> per node stream
When full	Rejects publish → publisher blocks (backpressure)	Evicts oldest (freshest wins; producer never blocks)
On processing failure	Redeliver up to <code>VF_MAX_RETRIES</code> (3), then dead-letter with the error attached	Drop
Delivery guarantee	At-least-once, loss-free	Best-effort, latency-bounded
Default join policy	Wait indefinitely (bounded by <code>max_pending</code>)	10 s timeout, then drop the group
Kubernetes workload	Job per node	Deployment (StatefulSet when partitioned); finite producers are Jobs

Table 3. One flow-type switch selects the retention/discard policy of every edge, and with it the pipeline's semantics. Section 9.5 measures both under overload.

Three mechanisms back the reliability claims. **Ack-after-process**: acknowledgment follows successful processing and publication, so a crash causes redelivery, not loss. **Content-derived message ids**: publish ids are derived from `(flow, run, producer, trace, seq)`, so the broker's publish-dedup window absorbs the duplicate emit that follows a crash-and-retry — at-least-once delivery without double emission. **Replica-safe end-of-stream**: EOS markers carry the emitting replica id on a dedicated subject; every replica of a downstream node observes them, drains its inputs, forwards its own marker, and exits — which is how the finite experiments in §9 terminate cleanly without any central coordinator. §9.9 exercises all three mechanisms at once by SIGKILLing a worker replica mid-flow: the flow completes with zero loss and zero duplicates.

5 Joins

A node with several parents needs its inputs grouped into one `process(*inputs)` call. The grouping rule is a per-node `JoinPolicy` with two modes:

- **trace** (default) — group by lineage. Every message carries the `trace_id` of the producer message it descends from, so branches of a diamond that re-converge are matched exactly. This is the right join when inputs share an upstream producer, and it is what allows a join to *scale*: `partition_by='trace_id'` routes both halves of a group to the same replica by key hash (§9.6).
- **time** — group by event time within `tolerance_ms`, for fusing streams from *independent* producers (multiple cameras, sensors) that share no lineage. Producers stamp `event_ts` via the runtime context; the stamp travels with every descendant message. `quorum=k` lets a timed-out group emit with at least k of N parents (missing ones passed as `None`), and `collect` windows deliver high-rate parents (e.g. a 500 Hz IMU against 50 fps cameras) as per-group lists rather than 1:1 partners (§9.7).

Incomplete groups are governed by `timeout_seconds` and a `missing` policy — `drop`, `wait`, or `error` (fail the partial inputs so they redeliver and eventually dead-letter) — with a `max_pending` cap that bounds buffering. Defaults follow the flow type: BATCH waits (completeness), REALTIME times out and drops (a lost sibling frame must not stall the join).

6 The wire format

Bytes that cross a process boundary use a self-describing envelope carrying routing identity (producer, flow, run, trace, seq, replica), an event timestamp, tracing spans, metadata, and the payload. Two encodings coexist deliberately:

- **v3 — msgpack** [30] (default emit): compact and fast for Python-to-Python edges; ndarrays use a raw buffer codec, everything else falls back to pickle.
- **v4 — protobuf** [29]: a language-neutral `videoflow.v1.Envelope` whose payload is a typed message — `Tensor` for arrays and frames, `Value` for structured data, any vendor proto by fully-qualified name. This is the encoding non-Python components implement, and a graph containing one forces v4 for the whole run; a mixed flow that would need pickle on the wire is a compile error.

The decoder auto-detects the version from the leading byte (a msgpack map and a protobuf field tag occupy disjoint ranges), so mixed-version tooling needs no out-of-band hint. Payloads above 512 KiB spill to an external blob store (Redis [26] by default, scheme-registered factories for others) and the envelope carries only a reference — the claim-check pattern from the enterprise-integration literature [27, 28]. NATS defaults to a 1 MB message cap [23], and a 1080p RGB frame is ~6.2 MB; §9.8 locates the exact payload size where inline transport stops working and quantifies what the spill costs. The normative contract — envelope fields, payload-type mappings, routing requirements, each with a stable requirement id — lives in `spec/PROTOCOL.md`, enforced by golden vectors replayed against every SDK; an observable wire change requires an RFC plus updated vectors.

7 Deployment

The `videoflow` CLI turns a graph module exposing `build_flow()` into a running system. `videoflow run-local` is the development twin of `deploy`: it resolves the solution's config template and prep hook, starts a dev NATS/Redis in Docker when none is listening (and tears down only what it started), and runs one subprocess per node. `videoflow deploy` compiles the graph — inside the solution image when the operator's machine lacks the graph's ML dependencies — builds and loads images for the detected cluster flavor (k3s, kind, minikube, Docker Desktop, generic remote), provisions broker streams up front, and renders plain-dict manifests [31, 32]: per-node workloads and ConfigMaps, a default-deny-except-broker NetworkPolicy, PodDisruptionBudgets, and optional KEDA scalers on broker lag [33]. Every automated step has an explicit override (`--image`, `--nats`, `--namespace`, `--dry-run`, `--render-only`).

Graph concept	Kubernetes realization
<code>nb_tasks=N</code>	N replicas of the node's Deployment (competing consumers)
<code>nb_tasks=N</code> , <code>partition_by=k</code>	StatefulSet — replicas need stable identities to own partitions
Finite producer / BATCH node	Job
<code>device_type='gpu'</code>	<code>nvidia.com/gpu</code> resource limit + GPU-pool nodeSelector + taint toleration; <code>--gpu-runtime-class</code> , <code>--gpu-mode shared</code> , MIG/time-sliced resource names for shared cards
Node <code>name</code>	DNS-1123-checked resource names, labels, subjects, log fields
Worker health	<code>/readyz</code> , <code>/healthz</code> , startup probe, Prometheus [34] <code>/metrics</code>

Table 4. How graph declarations map to cluster objects.

Language-agnostic components. A node need not be Python. A `component.yaml` descriptor — schema-validated, distributed as an OCI artifact next to the images it references — declares a component's params, I/O, device support, and runtime. The `component('oci://ghcr.io/acme/sort-tracker:1.2.0', params=...)` factory returns a remote node that subclasses the normal bases, so wiring, validation, scaling, and manifest generation are unchanged, while the vendor image speaks the v4 wire protocol out-of-process. Descriptors can be inspected without pulling multi-gigabyte ML images, and `cosign`-verified on pull.

8 Extension seams

Variant behavior is added by registration, not modification. Each seam follows one shape: a module-level registry seeded with the built-ins, an explicit `register_*`, and a lookup that raises `ValueError` naming the known values and the fix. Where nothing would import the providing package first, the registry consults an `importlib.metadata` entry-point group once on a miss — so a blob store named only in configuration still resolves.

Seam	Registration	Selected by
Blob store	<code>register_blob_store(scheme, factory)</code>	Blob URL scheme
Payload encoding (v4)	<code>register_payload_encoder</code> / <code>register_payload_type</code>	Python type / proto FQN
Cluster flavor	<code>register_cluster_flavor(handler)</code>	Cluster detection
GPU allocation mode	<code>register_gpu_mode(strategy)</code>	<code>--gpu-mode</code>
Config question type	<code>register_question_type(qtype, coercer)</code>	<code>x-questions</code> entry

Table 5. Extension seams. Equally deliberate are the abstractions *not* built: a transport layer below `Messenger/BlobStore` is spec-fixed (a second broker starts as an RFC, not a refactor), and an execution-engine registry waits until a third engine forces the lifecycle into the ABC.

9 Experimental evaluation

The design above promises specific, measurable behavior: a compact wire format with an escape hatch for large frames, near-linear stage scaling from a one-line change, per-hop overhead small enough for deep pipelines, a semantic switch between loss-free and freshest-wins delivery, joins that scale by partitioning, time-based fusion that degrades gracefully, a payload-placement policy that keeps large frames deliverable, and crash recovery without loss or duplication. This section tests each claim with deliberately minimal toy graphs, so the numbers measure the framework rather than any model.

9.1 Method and setup

All experiments run on one machine (Table 6) with the **local engine** — one OS subprocess per node replica, all traffic through a real NATS JetStream broker in Docker. This is the identical worker code path Kubernetes uses; what it does not measure is network latency between hosts, image pull, or scheduler behavior. Producers stamp each payload with a wall-clock t_0 at emission; the sink appends (i, t_0, t_1) records to a JSONL file. Since all processes share one clock, end-to-end latency is $t_1 - t_0$ with no cross-host skew. Broker-backed experiments use 3 repetitions per configuration (E7 sweeps 30 size×mode cells once each — it identifies regimes and a hard failure boundary, not fine differences); reported values are medians unless noted. The harness (~610 lines, `white-paper/experiments/`) and all raw results ship in the repository.

Component	Value
CPU / RAM	Intel Core i9-13900KF, 32 hardware threads / 94 GiB
OS / kernel	Ubuntu Linux, 6.8.0-124-generic
Python / Videoflow	3.12.13 / 1.0.0
Broker / blob store	NATS 2.10 (JetStream, Docker) / Redis 7 (Docker)
Key libraries	nats-py 2.15.0, msgpack 1.2.1, protobuf 7.35.1, numpy 2.5.1

Table 6. Experimental environment.

Exp	Question	Factors (levels)	Fixed	Responses
E1	Wire-format cost	version (v3, v4) × payload (4 shapes) × placement (inline, blob)	—	size, encode/decode time
E2	Stage scale-out	<code>nb_tasks</code> (1, 2, 4, 8)	240 msgs, 20 ms/msg stage	sink throughput
E3	Per-hop overhead	depth (1, 2, 4, 8)	250 msgs @ 50 msg/s	median / p95 latency
E4	Overload semantics	flow type (BATCH, REALTIME)	300 msgs @ 100 msg/s into 40 msg/s stage	delivered %, latency profile
E5	Join scale-out	joiner <code>nb_tasks</code> (1, 2, 4)	240 msgs, diamond, 20 ms join	throughput, correctness
E6	Quorum fusion	—	2×50 fps cams, one drops ⅓; tol 12 ms, quorum 1	group completeness
E7	Payload placement	size (1 KiB–6 MiB) × policy (inline, blob, auto)	producer → sink	delivered %, throughput
E8	Crash recovery	—	E2 graph, 2 replicas, one SIGKILLED at $t \approx 2.5$ s	loss, duplicates, recovery time

Table 7. Design of experiments. Graph shapes are shown in Figure 3.

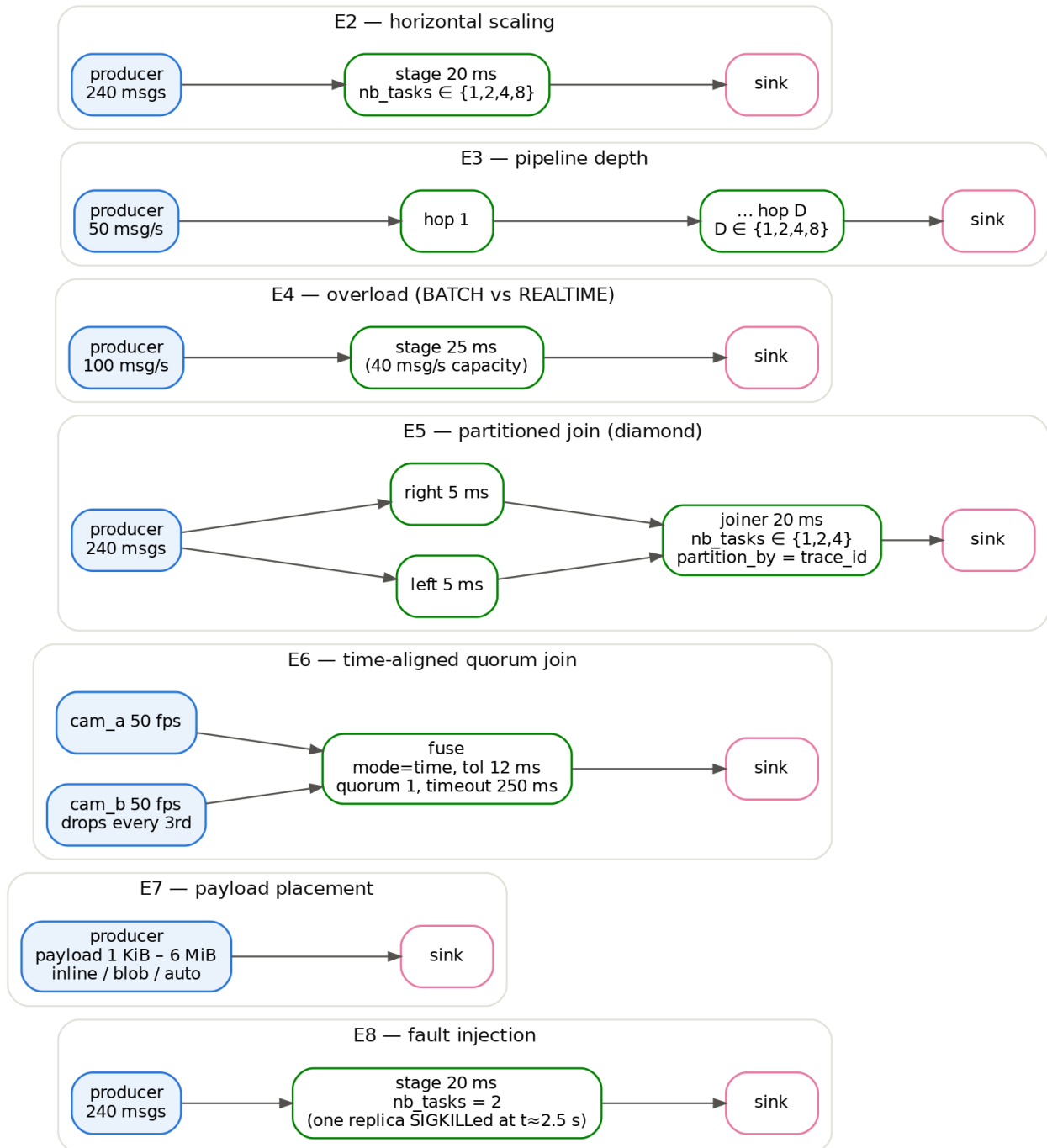


Figure 3. The toy graphs. Blue = producer, green = processor, pink = consumer.

9.2 E1 — wire-format microbenchmark

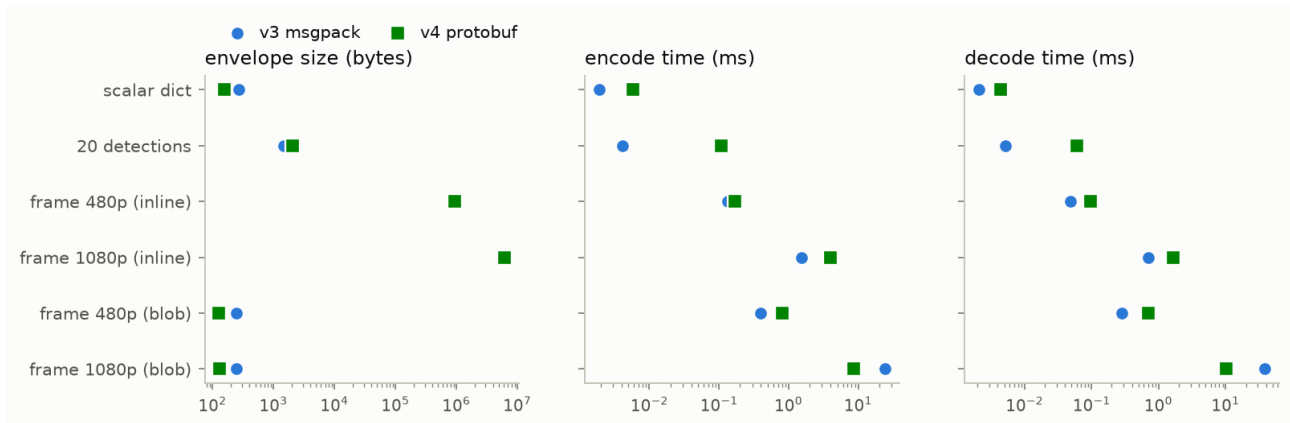


Figure 4. Envelope size and median encode/decode time (log axes) for msgpack v3 vs protobuf v4, inline vs blob-offloaded. Full numbers in Table 8.

Payload	Placement	v3 size (B)	v4 size (B)	v3 enc/dec (ms)	v4 enc/dec (ms)
scalar dict	inline	267	154	0.002 / 0.002	0.006 / 0.004
20 detections	inline	1,462	2,060	0.004 / 0.005	0.107 / 0.060
frame 480p	inline	921,816	921,679	0.132 / 0.048	0.169 / 0.096
frame 1080p	inline	6,221,016	6,220,881	1.513 / 0.700	3.985 / 1.644
frame 480p	blob	248	127	0.397 / 0.283	0.808 / 0.697
frame 1080p	blob	248	128	24.014 / 38.271	8.495 / 10.243

Table 8. E1 results (medians of 300 reps; frames 30 reps). Envelope size includes all routing/tracing fields; blob rows show the on-wire envelope with the payload in Redis, and their times include the Redis round trip on this machine's loopback.

Findings. For small structured payloads the v4 protobuf envelope is 42 % smaller than v3 (msgpack's per-key field names dominate), while for detection-style nested dicts v4 is moderately larger and slower — its generic `Value` tree trades compactness for language neutrality, which is exactly the intended trade. Frames dominate everything: a raw 1080p frame is ~6.2 MB — over NATS's 1 MB default cap — and the blob store collapses the on-wire envelope to 128–248 B regardless of frame size, at the cost of a store round trip (8–24 ms encode for 1080p on loopback Redis). Both codecs move frames at memory-bandwidth speeds when inline (≤ 4 ms encode for 1080p); none of these costs is remotely comparable to a model inference, so the wire is not the bottleneck the design should optimize further.

9.3 E2 — horizontal scaling of a stage

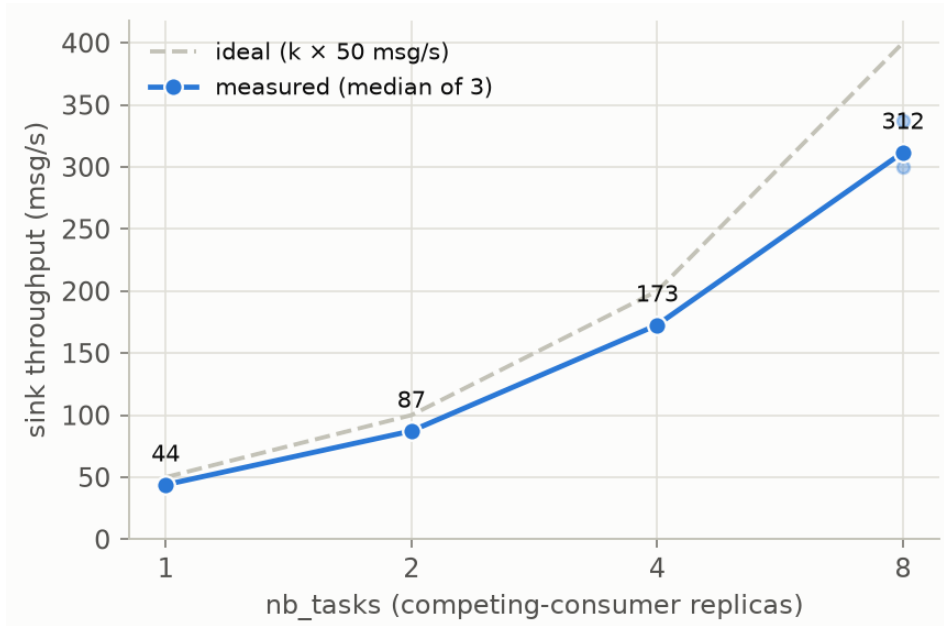


Figure 5. Sink throughput vs `nb_tasks` for a 20 ms/msg stage (240 messages, BATCH). Faint dots are individual repetitions; the dashed line is ideal linear scaling of the stage's 50 msg/s single-replica capacity.

nb_tasks	Throughput (msg/s)	Speedup	Efficiency vs ideal
1	44	1.0×	88%
2	87	2.0×	87%
4	173	3.9×	86%
8	312	7.1×	78%

Table 9. E2 medians. Efficiency is measured against $k \times 50$ msg/s.

Findings. Changing one constructor argument (`nb_tasks`) scales a stage to 7.1× at 8 replicas with no code or topology changes — replicas share a durable and compete for work. Efficiency declines gently (88 % at $k=1$ to 78 % at $k=8$): with 20 ms of work per message, per-message broker overhead (~2 ms, §9.4) is ~9 % of service time, and the single producer and single sink add serialization points at the ends. The practical guidance follows directly: replicate stages whose per-item cost dominates the per-hop overhead — precisely the model-inference stages replication exists for.

9.4 E3 — per-hop overhead

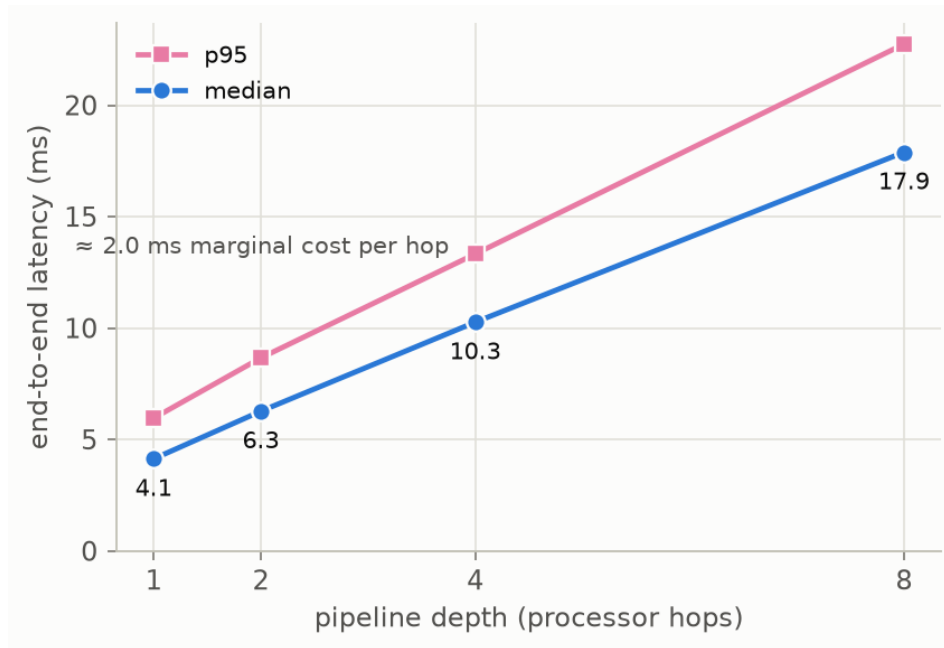


Figure 6. End-to-end latency vs pipeline depth for identity hops at 50 msg/s (no queueing). The fitted slope of the median is the marginal cost of one graph edge.

Depth	Median (ms)	p95 (ms)
1	4.1	6.0
2	6.3	8.7
4	10.3	13.3
8	17.9	22.7

Table 10. E3 latency medians across 3 reps.

Findings. Latency grows linearly with depth: ≈ 2.0 ms median per hop (publish, JetStream persistence, durable fetch, decode — on loopback), with p95 tracking the median at a near-constant offset — the pipeline adds overhead but little jitter. A ten-stage pipeline therefore costs ~ 20 ms of transport latency end-to-end, comfortably inside a 33 ms frame budget at 30 fps, and the linearity means there is no superlinear penalty for factoring logic into more, smaller nodes.

9.5 E4 — delivery semantics under overload

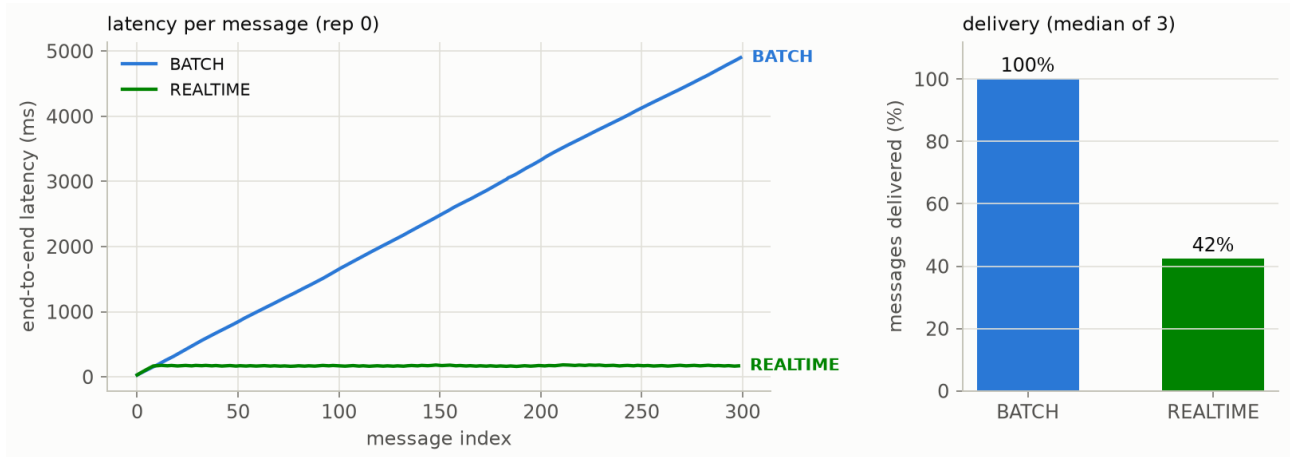


Figure 7. The same overloaded graph (100 msg/s into a 40 msg/s stage) run BATCH and REALTIME. Left: per-message latency. Right: fraction of the 300 messages that reached the sink.

Flow type	Delivered	Median latency (ms)	Max latency (ms)	Makespan (s)
BATCH	300 / 300 (100%)	2472	4900	8.3
REALTIME	127 / 300 (42%)	171	185	3.5

Table 11. E4 medians across 3 reps. Makespan is sink-side (first to last delivery).

Findings. The flow-type switch behaves exactly as specified. BATCH delivers 100 % under 2.5× overload — interest retention applies backpressure and the backlog drains at stage capacity — at the price of latency that grows linearly to ~4.9 s for the last message: correct for offline processing of recorded video, where completeness matters and latency does not. REALTIME's one-message stream buffer delivers the freshest 42 % ($\approx \text{stage capacity} \div \text{producer rate}$) with latency pinned at ≈ 171 ms regardless of how long the overload lasts: correct for live monitoring, where a stale detection is worthless. The experiment also exercises graceful termination in both modes — EOS propagates through the overloaded stage and both flows exit cleanly without a coordinator.

9.6 E5 — scaling a join by partitioning

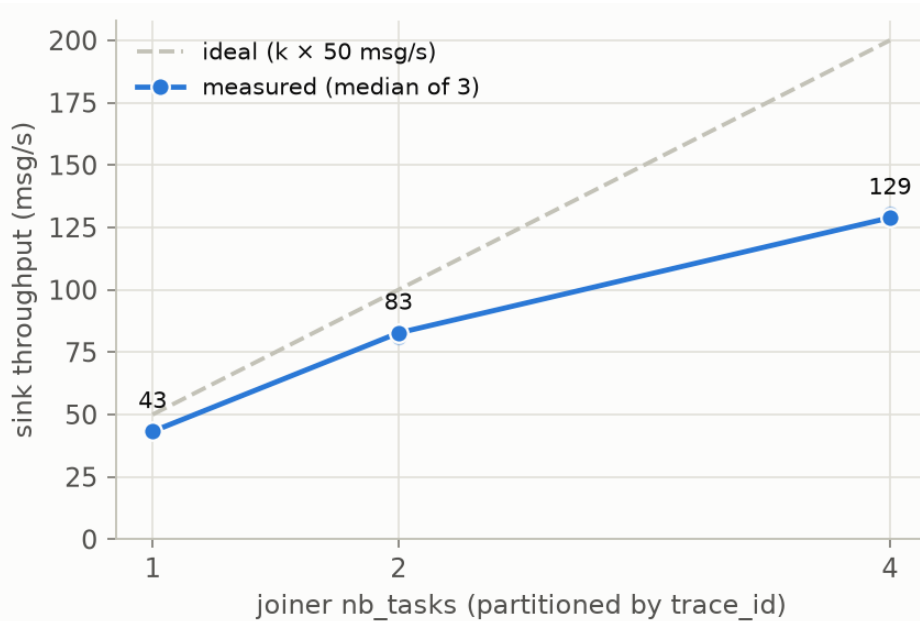


Figure 8. Sink throughput of the diamond graph vs joiner `nb_tasks` with `partition_by='trace_id'` (240 messages, 20 ms join).

nb_tasks	Throughput (msg/s)	Speedup	Groups correct
1	43	1.0×	720 / 720
2	83	1.9×	720 / 720
4	129	3.0×	720 / 720

Table 12. E5 medians. "Groups correct" counts sink records whose two halves carry the same message index — i.e. joins that matched the right partners.

Findings. A join is normally a serialization point — every parent's half must reach the same worker. Partitioning by `trace_id` removes it: each replica sees every message (per-replica durables) but assembles only the groups it owns, and both halves of a lineage hash to the same owner by construction. Throughput scales to 3.0× at 4 replicas with **every group correctly matched in every run** (2160 of 2160 across all runs) — no misrouted or partial joins, which is the property that matters when the join carries per-object state.

9.7 E6 — time-aligned fusion with a quorum

Rep	Groups emitted	Both cameras	cam_a only	cam_b only
0	150	100	50	0
1	150	100	50	0
2	150	100	50	0

Table 13. E6: two independent 50 fps producers (no shared lineage), `cam_b` dropping every third frame; time-mode join with 12 ms tolerance, 250 ms timeout, quorum 1, `REALTIME`. Expected per rep: ~100 complete groups and ~50 quorum groups.

Findings. Trace-mode joins cannot fuse these streams at all — the producers share no lineage, so no group would ever complete. Time-mode grouping matches frames whose event timestamps fall within tolerance, and the quorum turns

cam_b's dropped frames from a stall into a degraded-but-on-time emission with the missing side passed as `None` : 67 % of groups fused both cameras (matching cam_b's $\frac{2}{3}$ survival rate) and the rest emitted on timeout with cam_a alone. This is the designed failure mode for multi-camera rigs: a flaky source degrades its own contribution, never the pipeline's cadence.

9.8 E7 — payload placement: when is the blob store needed?

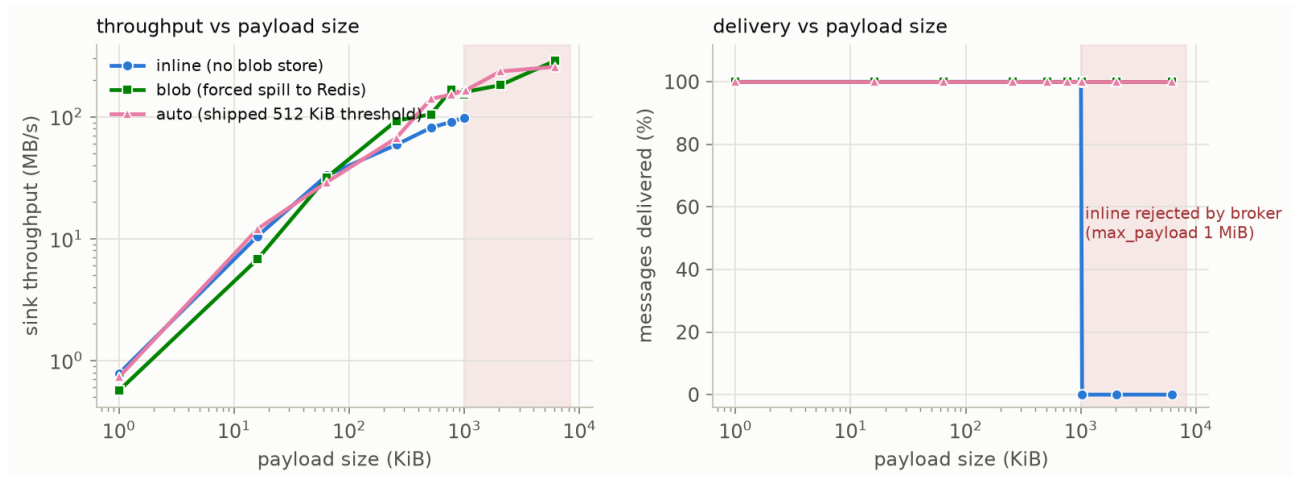


Figure 9. A producer → sink flow (BATCH) sweeping payload size under three placement policies. Left: sink throughput (log-log) over the sizes each policy completed. Right: the delivery cliff — inline transport goes from 100 % to 0 % between two adjacent sizes. The shaded region is where the envelope exceeds the broker's 1 MiB `max_payload`.

Payload	Inline delivered	Inline MB/s	Blob MB/s	Auto MB/s
1 KiB	150 / 150	1	1	1
16 KiB	150 / 150	10	7	12
64 KiB	150 / 150	33	32	29
256 KiB	150 / 150	59	93	67
512 KiB	60 / 60	81	105	140
768 KiB	60 / 60	91	166	152
1,000 KiB	60 / 60	98	153	163
1 MiB	0 / 60	rejected	160	164
2 MiB	0 / 30	rejected	182	235
6 MiB	0 / 30	rejected	288	257

Table 14. E7 results. *Inline*: payload rides in the broker message, spill disabled. *Blob*: every payload takes the Redis round trip. *Auto*: the shipped policy — inline below the 512 KiB threshold, blob above. Blob and auto delivered 100 % at every size.

Findings. The breaking point is exactly where the arithmetic says it must be, and it is a cliff, not a slope. A 1,024,000 B payload delivers 100 %; a 1,048,576 B payload delivers zero — the ~300 B envelope pushes it past NATS's 1 MiB `max_payload` [23], the broker rejects the publish (`MaxPayloadError`), and the producer worker dies mid-flow. An oversized payload is a hard runtime failure, which is why payload placement is a framework policy rather than a user responsibility. The cap itself is not a NATS quirk: Kafka ships the same ~1 MB default limit [24, 25], and both ecosystems' guidance for larger payloads is exactly the claim-check pattern [27, 28] that the blob store implements — store the bytes externally, send a reference. Two quantitative surprises sharpen the picture. First, the blob path is not a

tax at scale: above ~256 KiB it is *faster* than inline (288 vs 98 MB/s at their respective maxima), because JetStream persists every inline message to its file-backed stream while the blob path moves bytes through in-memory Redis and gives the broker only a ~130-byte reference to store. Second, below ~64 KiB the two paths are indistinguishable, so the shipped 512 KiB threshold sits comfortably inside the regime where spilling costs nothing and safely below the size where not spilling is fatal. The practical rule for video: metadata and detections ride inline; any raw frame beyond ~480p must go through the blob store — and *auto* makes that decision per message.

9.9 E8 — crash recovery: SIGKILL a replica mid-flow

Rep	Delivered	Unique	Duplicates	Killed at (s)	Total wall (s)
0	240 / 240	240	0	2.5	66
1	240 / 240	240	0	2.5	66

Table 15. E8: the E2 graph with two competing stage replicas; replica 0 is SIGKILLed ~2.5 s into the run — a hard crash with no chance to ack, publish EOS, or clean up. A clean two-replica run of the same graph completes in ~10 s.

Findings. Every run delivered all 240 messages with 0 duplicates. The three reliability mechanisms of §4 are each visible in the outcome. Messages the dead replica had fetched but not acked were *redelivered* to the surviving replica after the 60 s `ack_wait` — ack-after-process turned the crash into a delay (~66 s total wall clock), not a loss. Messages the dead replica had processed and published but not yet acked were reprocessed and republished by the survivor — and the sink still saw no duplicates, because the republish carries the same content-derived message id and JetStream's publish-dedup window absorbs it. And the flow still *terminated*: EOS completion needs only one surviving replica's marker per parent, so the killed replica's missing EOS did not hang the sink. The recovery bound is explicit and tunable — worst-case added latency after a crash is one `ack_wait`, traded against the risk of spurious redelivery to slow-but-alive workers.

9.10 Threats to validity

- **Single machine, loopback broker.** Per-hop latency and blob-store round trips will be higher over a real network; the scaling and semantics results (which depend on routing and retention, not link speed) should transfer. The Kubernetes engine additionally pays scheduling and image-pull costs not measured here.
- **Sleep-based service times.** Stages simulate work with `time.sleep`, which scales past physical cores without CPU contention. This isolates framework routing overhead — the quantity under test — but means E2/E5 efficiency numbers are upper bounds for genuinely CPU-bound stages at high replica counts on smaller machines (GPU-bound stages, the primary replication target, behave like sleeps from the scheduler's perspective).
- **Toy payloads.** Broker-backed experiments carry ~100-byte dicts; E1 bounds the additional per-message cost of realistic frames (≤ 4 ms encode, ~1 ms blob write for 480p).
- **Scale of repetition.** 3 reps per cell with low dispersion (visible as the point spread in Figures 5 and 8); the trends are unambiguous, the third significant figure is not. E7 runs each cell once — its findings are regime boundaries (a delivery cliff, a throughput crossover), which single runs identify reliably; its per-cell latency and throughput values carry run-to-run noise.
- **One fault model.** E8 injects a hard worker crash (SIGKILL). It does not exercise broker failure, network partitions, or slow-worker gray failures; the recovery bound measured is specific to worker loss under a healthy broker.

10 Related work

Video analytics systems. A line of systems from the video-analytics literature sits *above* the layer Videoflow occupies: they assume a vision pipeline exists and optimize how it runs. VideoStorm [1] schedules thousands of concurrent

queries across a cluster using offline resource–quality profiles, and explicitly distinguishes itself from general stream processors on the grounds that they allocate resources fairly rather than by output quality and lag — the same separation of concerns this paper draws from the other side. Chameleon [2] continuously re-picks pipeline knobs (resolution, frame rate, model) at runtime; Ekya [6] jointly schedules inference and continuous retraining on shared edge GPUs; VideoEdge [7] places pipeline stages across a camera–edge–cloud hierarchy. All are complementary controllers and schedulers: their pipelines need a substrate that runs DAG stages as scalable workers, which is precisely what Videoflow provides. NoScope [4] and Focus [3] accelerate *queries over recorded video* with cascades of cheap specialized models and approximate ingest-time indexes — a different design point from live dataflow. Scanner [5] is the closest video system in spirit: computations are dataflow graphs over compressed video tables, but it targets throughput-oriented batch analysis of stored video on tightly-coupled clusters, where Videoflow targets continuously running, independently scalable stream pipelines.

General stream processors. Videoflow's DAG-of-operators lineage runs through the scale-out stream engines — Storm [8], Spark Streaming's discretized streams [9], MillWheel [10], Samza [11], Naiad's timely dataflow [12], and Flink [13, 14]; the SIGMOD tutorial by Carbone et al. [15] surveys the family. The sharpest architectural contrast is with how these engines place computation. Flink parallelizes each operator into subtasks connected by partitioned in-process streams and achieves exactly-once state via asynchronous barrier snapshotting [13, 14]; Hazelcast Jet [17] goes further and replicates the *entire* graph on every core precisely to avoid crossing machine boundaries between operators. Videoflow deliberately takes the opposite position — one worker (process, container) per node, with a persistent broker on every edge — accepting ~2 ms per hop (§9.4) to gain properties the engine-internal designs lack: nodes deploy, scale, fail, and get GPU-scheduled independently as ordinary Kubernetes workloads; every edge has externally visible retention, replay, and a dead-letter queue; and consumer lag is a first-class autoscaling signal [33]. Its consistency stance also differs: rather than checkpoint-based exactly-once state [14], Videoflow composes at-least-once delivery, content-derived publish dedup, and opt-in idempotent sinks (§4, validated in §9.9) — a fit for vision pipelines, whose per-frame state is usually reconstructible while side effects need sink-level dedup anyway. Samza [11] is the closest relative in data plane — operators decoupled by a persistent log (Kafka) — while Ray [16] marks the opposite end of the scheduling spectrum, a dynamic task graph constructed at runtime versus Videoflow's compile-then-deploy static DAG.

Media pipelines and serverless video. GStreamer [18] popularized the graph-of-media-components model in a single process, and NVIDIA DeepStream [19] extends it with GPU-accelerated inference plugins on one machine; Videoflow lifts the same mental model across machines, and its language-agnostic OCI components (§7) play the role plugins play there. ExCamera [20] and Sprocket [21] burst-parallelize video work across thousands of short-lived serverless functions — the right shape for one-shot transformations of stored video, where Videoflow's long-running workers with warm models target continuous streams.

11 Conclusion

Videoflow demonstrates that a distributed video-processing system can keep a single-machine programming model — plain Python classes wired into a DAG — by pushing all distribution concerns into conventions checked at build time and topology derived from names. The experiments confirm the design's quantitative claims on real broker infrastructure: ~2 ms per graph edge, near-linear stage and join scale-out from one-line changes, a delivery-semantics switch that trades completeness against bounded latency exactly as specified, fusion policies that degrade gracefully under source failure, a payload-placement policy that steps over the broker's message-size cliff (and above ~256 KiB outperforms inline transport), and crash recovery that turns a SIGKILLED worker into a bounded delay with zero loss and zero duplicates. The protobuf wire contract and OCI component distribution extend the same model beyond Python, toward a marketplace of interoperable components with the Python worker as the executable reference implementation.

References

- [1] H. Zhang, G. Ananthanarayanan, P. Bodík, M. Philipose, P. Bahl, and M. J. Freedman. Live Video Analytics at Scale with Approximation and Delay-Tolerance. In *14th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, pp. 377–392, 2017.
- [2] J. Jiang, G. Ananthanarayanan, P. Bodík, S. Sen, and I. Stoica. Chameleon: scalable adaptation of video analytics. In *ACM SIGCOMM*, pp. 253–266, 2018.
- [3] K. Hsieh, G. Ananthanarayanan, P. Bodík, S. Venkataraman, P. Bahl, M. Philipose, P. B. Gibbons, and O. Mutlu. Focus: Querying Large Video Datasets with Low Latency and Low Cost. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pp. 269–286, 2018.
- [4] D. Kang, J. Emmons, F. Abuzaid, P. Bailis, and M. Zaharia. NoScope: Optimizing Neural Network Queries over Video at Scale. *Proceedings of the VLDB Endowment*, 10(11), pp. 1586–1597, 2017.
- [5] A. Poms, W. Crichton, P. Hanrahan, and K. Fatahalian. Scanner: Efficient Video Analysis at Scale. *ACM Transactions on Graphics*, 37(4) (SIGGRAPH), 2018.
- [6] R. Bhardwaj, Z. Xia, G. Ananthanarayanan, J. Jiang, N. Karianakis, Y. Shu, K. Hsieh, P. Bahl, and I. Stoica. Ekya: Continuous Learning of Video Analytics Models on Edge Compute Servers. In *19th USENIX NSDI*, pp. 119–135, 2022.
- [7] C.-C. Hung, G. Ananthanarayanan, P. Bodík, L. Golubchik, M. Yu, P. Bahl, and M. Philipose. VideoEdge: Processing Camera Streams using Hierarchical Clusters. In *IEEE/ACM Symposium on Edge Computing (SEC)*, pp. 115–131, 2018.
- [8] A. Toshniwal et al. Storm@twitter. In *ACM SIGMOD*, pp. 147–156, 2014.
- [9] M. Zaharia, T. Das, H. Li, T. Hunter, S. Shenker, and I. Stoica. Discretized Streams: Fault-Tolerant Streaming Computation at Scale. In *24th ACM Symposium on Operating Systems Principles (SOSP)*, pp. 423–438, 2013.
- [10] T. Akidau et al. MillWheel: Fault-Tolerant Stream Processing at Internet Scale. *Proceedings of the VLDB Endowment*, 6(11), pp. 1033–1044, 2013.
- [11] S. A. Noghbi et al. Samza: Stateful Scalable Stream Processing at LinkedIn. *Proceedings of the VLDB Endowment*, 10(12), pp. 1634–1645, 2017.
- [12] D. G. Murray, F. McSherry, R. Isaacs, M. Isard, P. Barham, and M. Abadi. Naiad: A Timely Dataflow System. In *24th ACM SOSP*, pp. 439–455, 2013.
- [13] P. Carbone, A. Katsifodimos, S. Ewen, V. Markl, S. Haridi, and K. Tzoumas. Apache Flink: Stream and Batch Processing in a Single Engine. *IEEE Data Engineering Bulletin*, 38(4), pp. 28–38, 2015.
- [14] P. Carbone, S. Ewen, G. Fóra, S. Haridi, S. Richter, and K. Tzoumas. State Management in Apache Flink: Consistent Stateful Distributed Stream Processing. *Proceedings of the VLDB Endowment*, 10(12), pp. 1718–1729, 2017.
- [15] P. Carbone, M. Fragkoulis, V. Kalavri, and A. Katsifodimos. Beyond Analytics: The Evolution of Stream Processing Systems. In *ACM SIGMOD*, pp. 2651–2658, 2020.
- [16] P. Moritz, R. Nishihara, S. Wang, A. Tumanov, R. Liaw, E. Liang, M. Elibol, Z. Yang, W. Paul, M. I. Jordan, and I. Stoica. Ray: A Distributed Framework for Emerging AI Applications. In *13th USENIX OSDI*, 2018.
- [17] C. Gencer, M. Topolnik, V. Ďurina, et al. Hazelcast Jet: Low-latency Stream Processing at the 99.99th Percentile. *Proceedings of the VLDB Endowment*, 14(12), pp. 3110–3121, 2021.
- [18] GStreamer Team. GStreamer: open source multimedia framework. gstreamer.freedesktop.org.
- [19] NVIDIA. NVIDIA DeepStream SDK Developer Guide. docs.nvidia.com/metropolis/deepstream/dev-guide/.
- [20] S. Fouladi et al. Encoding, Fast and Slow: Low-Latency Video Processing Using Thousands of Tiny Threads. In *14th USENIX NSDI*, pp. 363–376, 2017.
- [21] L. Ao, L. Izhikevich, G. M. Voelker, and G. Porter. Sprocket: A Serverless Video Processing Framework. In *ACM Symposium on Cloud Computing (SoCC)*, pp. 263–274, 2018.
- [22] NATS Authors. JetStream. *NATS Documentation*, docs.nats.io/nats-concepts/jetstream.
- [23] NATS Authors. Configuring NATS Server: `max_payload` (default 1 MB; settable to 64 MB, ≤8 MB recommended). *NATS Documentation*, docs.nats.io/running-a-nats-service/configuration.
- [24] J. Kreps, N. Narkhede, and J. Rao. Kafka: a Distributed Messaging System for Log Processing. In *NetDB Workshop*, 2011.
- [25] Apache Kafka Documentation. Broker configs: `message.max.bytes` (default 1,048,588 B). kafka.apache.org/documentation; Confluent, Kafka Message Size Limit, confluent.io/learn/kafka-message-size-limit.
- [26] Redis. redis.io.
- [27] G. Hohpe and B. Woolf. *Enterprise Integration Patterns: Designing, Building, and Deploying Messaging Solutions*. Addison-Wesley, 2003. Pattern: Claim Check.

- [28] Microsoft. Claim-Check pattern. *Azure Architecture Center*, learn.microsoft.com/azure/architecture/patterns/claim-check.
- [29] Google LLC. Protocol Buffers Documentation. protobuf.dev.
- [30] S. Furuhashi et al. MessagePack Specification. github.com/msgpack/msgpack (msgpack.org).
- [31] A. Verma, L. Pedrosa, M. Korupolu, D. Oppenheimer, E. Tune, and J. Wilkes. Large-scale cluster management at Google with Borg. In *ACM EuroSys*, 2015.
- [32] B. Burns, B. Grant, D. Oppenheimer, E. Brewer, and J. Wilkes. Borg, Omega, and Kubernetes. *ACM Queue*, 14(1), pp. 70–93, 2016.
- [33] KEDA Authors. KEDA — Kubernetes Event-driven Autoscaling. keda.sh (CNCF Graduated project).
- [34] Prometheus Authors. Prometheus — Monitoring system and time series database. prometheus.io (CNCF Graduated project).
- [35] J. de Armas. Videoflow, 2019 — github.com/videoflow/videoflow. The original single-process release; this paper describes the distributed successor. The normative protocol contract is `spec/PROTOCOL.md`; the experiment harness and raw results are `white-paper/` in the same repository.